

Sajana IELTS 2.0

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Document Version: 1.0.0

Date: June 2026

Target Stack: Next.js, PostgreSQL, Prisma ORM, Sanity CMS

1. Introduction & Project Goals

1.1 Purpose

This document specifies the functional, non-functional, business, and architectural requirements for Sajana IELTS 2.0. This platform serves as a modern, high-performance web presence for a premium English language academy, streamlining student lead generation and content distribution.

1.2 Business & Project Goals

- **Promote IELTS Services:** Market elite educational packages tailored for IELTS Academic, IELTS General, and Spoken English.
- **Content Hub:** Host high-quality, searchable preparation materials (Cue cards, essays, vocabulary) to maximize organic search rankings.
- **Lead Acquisition Engine:** Automate the collection of target-driven student leads to convert traffic into active paying enrollments.
- **Academic & Job Portfolio Asset:** Act as a clean, highly structured reference codebase demonstrating master-level system design principles for MSc applications and software engineering evaluation tracks.

2. System Scope & Boundaries

2.1 Visual Color & Identity System

To reflect a premium educational institution, the interface adheres to a strict visual design palette:

- **Primary Background:** Pure White (#FFFFFF)
- **Primary Brand Tone:** Deep Blue (#0F172A) — used to build authority, trust, and deep typography contrast.
- **Accent Core Highlight:** Gold Accent (#D4AF37) — used exclusively for active Call-to-Action buttons, focus targets, and validation badges.

2.2 Explicit Site Architecture

Sajana IELTS 2.0 restricts public page availability strictly to the following target layouts:

1. **Home Page:** Hero banner, Teacher Introduction, Courses Overview, Student Results Carousel, Latest Blog Posts, Testimonials, Contact CTA.
2. **About Teacher:** Extensive bio, certified credentials, historical professional background, score tracking sheets.
3. **IELTS Courses:** Fixed technical data sheets tracking parameters for Academic, General, and Spoken English (including Duration, Fee, and Weekly Schedule options).
4. **IELTS Resources:** Grouped landing directories parsing nested categories: Speaking, Writing, Reading, Listening, Vocabulary.
5. **Blog:** Paginated index surfaces parsing real-time headless CMS documents.
6. **Student Results:** Historical band grids sorting raw student achievements and validation reviews.
7. **Contact:** Explicit communication form containing immediate social entry configurations (WhatsApp click-to-chat API, Facebook profile links, direct email links).

3. Detailed Requirements Spec

3.1 Functional Requirements (FR)

FR-1: Lead Management Engine

Every public view layer must feature a persistent "Book Free Demo" activation module. The form must validate and capture standard data parameters: Student Name, Phone Number, Email Address, Target Band Score (Dropdown selector matching increments of 0.5 within limits of 4.0 to 9.0), and an optional custom Message. All generated inquiries must bypass public view layers and write directly into a local PostgreSQL transactional schema pool via Prisma ORM execution layers.

FR-2: Decoupled CMS Integration

Instructors must be able to inject rich text resources—including Speaking Cue Cards, Sample Essays, and Vocabulary Modules—remotely via a decoupled Sanity Studio interface, eliminating the need to modify source files or trigger deployment pipelines. Sanity document collections must be automatically distributed across corresponding UI routes matching their respective categories.

FR-3: Restricted Administrator Space

Features a simple, secure authentication gate (/admin or /studio) protected by token encryption validation layers. Authenticated administrators can perform complete CRUD actions (Create, Read, Update, Delete) on student feedback entries and test performance charts.

3.2 Non-Functional Requirements (NFR)

NFR-1: Search Engine Optimization (SEO) Architecture

Standardize URL generations using clean semantic patterns rather than database row primary IDs (e.g., /ielts-speaking-cue-card-2026, /ielts-writing-task-2-opinion-essay, /ielts-vocabulary-for-education). Next.js metadata compilation loops must automatically inject dynamic Meta Titles, Descriptions, robots.txt instructions, and a programmatically updated sitemap.xml file. Build interactive validation scripts using JSON-LD schemas mapped directly onto courses and resource pages to capture high-visibility Google Rich Snippets.

NFR-2: Accessibility Framework (WCAG 2.1 AA Compliance)

Every input field must declare an explicit markup label tag element. Focus highlights must follow standard patterns (focus:ring-2 focus:ring-[#D4AF37] focus:outline-none), allowing users to navigate via keyboard tabs without relying on mouse pointers. Form structures, drop-down wrappers, and asynchronous button triggers must integrate descriptive ARIA tags (aria-label, aria-expanded).

NFR-3: Operational & Performance Bounds

Core content directories must utilize Incremental Static Regeneration (ISR) to maintain a Time-to-First-Byte benchmark of $t \leq 200\text{ms}$. All layouts must employ mobile-first media query breakpoints via Tailwind CSS to ensure rendering fidelity across diverse devices.

4. Advanced System Design & Database Schema

USER	
id	: String (UUID PK)
email	: String (Unique)
passwordHash	: String
name	: String
role	: Enum (ADMIN)

LEAD	
id	: String (UUID PK)
name	: String
email	: String
phone	: String (E.164)
targetBand	: String
message	: Text (Nullable)

STUDENT_RESULT	
id	: String (UUID PK)
studentName	: String
overallBand	: Float
listeningBand	: Float (Nullable)
readingBand	: Float (Nullable)
writingBand	: Float (Nullable)
speakingBand	: Float (Nullable)
feedback	: Text

TESTIMONIAL	
id	: String (UUID PK)
studentName	: String
courseTaken	: String
bandScore	: Float
reviewText	: Text
isFeatured	: Boolean

5. Deployment Architecture Blueprint

The live software architecture operates entirely on high-availability, production-grade zero-cost tiers:

- **Frontend & Deployment Engine:** Vercel Hobby Tier. Connects directly to the GitHub branch core engine to deliver atomic deployments and fast global asset distribution.
- **Relational Storage Core:** Neon / Supabase Serverless Postgres. Provides zero-scale sleeping operations, standard cloud database connection strings, and automated data security encryption.
- **Headless Document Asset Lake:** Sanity.io Free Developer Configuration. Supports real-time dashboard interactions for non-technical editors and global edge asset caching for resource files.